Backend Audit of cynbit.in

Prepared by: Dharna Gunidia
Assessment: Analyze Cynbit From a Backend Perspective
Date: 1 November 2026

1. About this report

I looked at the Cynbit website (https://www.cynbit.in) the way a normal visitor would and tried to understand how the backend probably works, where it could break, and what I would change. I opened the home, contact, careers, story and team pages, and the submission portal linked in the assessment instructions.

I did not try to break into anything, send automated or repeated requests, or touch any system that is not public. I only did passive checks, as the task asked.

Each finding below says whether I **saw it directly** or whether it is something I **still need to confirm**. A visitor cannot see server code, so some points are risks based on what is visible. I have kept those separate on purpose so that nothing is presented as a fact when it is not.

2. What the website looks like from outside

- The pages look server rendered, and every page has its own title and meta tags. A meta tag (`next-size-adjust`) makes me think the site is built with Next.js. I have not confirmed this.
- Besides the main pages there are dynamic routes such as `/careers/<role-name>` (20 openings), `/bitelevate/paths` and `/diaries`. These must be coming from a CMS or a database.
- There are two places where a visitor can send data to the backend: the **contact form** (name, email, company, industry, message) and the **newsletter signup** in the footer.
- The company has several products (Cynbit Studio, Revio, TenantEase, BitElevate). They probably have their own logins and databases.
- Two email addresses and three phone numbers are shown in plain text on every page.

3. Summary of findings

ID	Area	Finding	Severity	Status
S1	Security	Contact and newsletter forms are open to anyone	Medium	Need to confirm
S2	Security	Security headers and cookie settings unknown	Medium	Need to confirm
S3	Security	Career pages might expose closed or draft roles	Low	Need to confirm
S4	Security	Emails and phone numbers easy to scrape	Low	Seen on page
S5	Security	Submission portal not rated by a browser security tool	Low to Medium	Seen in browser
D1	Data / SEO	Wrong canonical and og:url on pages	Low	Seen on page
D2	Data / SEO	Share image is an SVG	Low	Seen on page

ID	Area	Finding	Severity	Status
D3	Data	Live-looking numbers may be fixed text	Low to Medium	Need to confirm
P1	Performance	Client logo list repeated four times in the HTML	Low	Seen on page
P2	Performance	Caching on careers and blog not known	Low	Need to confirm
E1	Error handling	No health or status endpoint found	Low	Need to confirm
A1	Authenticati on	Login cookie scope across products	Info	Need to confirm

4. Findings and fixes

S1. Contact and newsletter forms are open to anyone

What I noticed: Both forms can be used by any visitor without logging in. From outside I cannot tell what the server does with the data.

Why it matters: If there is no limit or validation, someone can fill the inbox with spam, send fake or very long input, or try to inject text into the notification email. The form also collects names and emails, so that data needs to be stored safely.

What I would check: Submit one normal test message, open the Network tab and note the request URL, the response status, and whether any bot protection or token is sent.

Fix: Validate every field on the server, add a hidden honeypot field, rate limit by IP, check the request origin, and save the message to the database before sending the email (use a queue so a mail failure does not lose the lead).

```
// app/api/inquiry/route.ts
import { NextResponse } from "next/server";
import { z } from "zod";

const Inquiry = z.object({
  name: z.string().trim().min(2).max(80),
  email: z.string().trim().email().max(254),
  company: z.string().trim().max(120).optional(),
  industry: z.string().trim().max(80).optional(),
  message: z.string().trim().min(10).max(2000),
  website: z.string().max(0).optional(), // honeypot, real users leave it empty
});

// simple in-memory limiter, use Redis if there is more than one server
const hits = new Map<string, { count: number; reset: number }>();
function tooMany(ip: string, max = 5, windowMs = 10 * 60 * 1000) {
  const now = Date.now();
  const h = hits.get(ip);
  if (!h || h.reset < now) {
    hits.set(ip, { count: 1, reset: now + windowMs });
    return false;
  }
  h.count += 1;
  return h.count > max;
}

export async function POST(req: Request) {
  const origin = req.headers.get("origin");
  if (origin && !["https://www.cynbit.in", "https://cynbit.in"].includes(origin)) {
    return NextResponse.json({ error: "forbidden" }, { status: 403 });
  }

  const ip = req.headers.get("x-forwarded-for")?.split(",")[0]?.trim() ?? "unknown";
  if (tooMany(ip)) {
```

```
    return NextResponse.json({ error: "too_many_requests" }, { status: 429 });
  }

  const parsed = Inquiry.safeParse(await req.json().catch(() => null));
  if (!parsed.success) {
    return NextResponse.json({ error: "validation_failed" }, { status: 422 });
  }
  if (parsed.data.website) {
    return NextResponse.json({ ok: true }, { status: 202 }); // bot, ignore quietly
  }

  // 1. save to database  2. add a job to send the notification email
  return NextResponse.json({ ok: true }, { status: 202 });
}
```

For the newsletter I would also add double opt-in (confirmation email) so nobody can sign up someone else's address.

S2. Security headers and cookie settings

What I noticed: I could not see the response headers from the page content, so I do not know which ones are set.

Why it matters: Missing headers make clickjacking, content sniffing and some script injection attacks easier. Cookies without `HttpOnly`, `Secure` and `SameSite` can be stolen or misused.

What I would check: Run `curl -sI https://www.cynbit.in/` once and look for `Strict-Transport-Security`, `Content-Security-Policy`, `X-Content-Type-Options`, `Referrer-Policy`, and anything that shows the server software (`Server`, `X-Powered-By`).

Fix:

```
// next.config.js
const securityHeaders = [
  { key: "Strict-Transport-Security", value: "max-age=63072000; includeSubDomains; preload" },
  { key: "X-Content-Type-Options", value: "nosniff" },
  { key: "Referrer-Policy", value: "strict-origin-when-cross-origin" },
  { key: "Permissions-Policy", value: "camera=(), microphone=(), geolocation=()" },
  {
    key: "Content-Security-Policy",
    value:
      "default-src 'self'; img-src 'self' data: https:; script-src 'self' 'unsafe-inline'; " +
      "style-src 'self' 'unsafe-inline'; frame-ancestors 'none'; base-uri 'self'; form-action 'self'",
  },
];

module.exports = {
  poweredByHeader: false,
  async headers() {
    return [{ source: "/(.*)", headers: securityHeaders }];
  },
};
```

The CSP above is a starting point. It would need testing against the analytics and font scripts the site uses.

S3. Career pages and unknown URLs

What I noticed: Each job has its own page at `/careers/<role-name>`, so the server looks the role up by name.

Why it matters: If an unknown name returns an error page with details, or if closed or draft roles can still be opened by guessing the name, information leaks.

What I would check: Open one clearly wrong URL such as `/careers/this-does-not-exist` and see whether the status is a clean 404.

Fix: Return a real 404 for anything that is not an open role, and generate the pages from the list of open roles only.

```
// app/careers/[slug]/page.tsx
import { notFound } from "next/navigation";

export const revalidate = 3600;

export async function generateStaticParams() {
  const roles = await getOpenRoles();
  return roles.map((r) => ({ slug: r.slug }));
}

export default async function Role({ params }: { params: { slug: string } }) {
  const role = await getRole(params.slug); // only returns roles with status "open"
  if (!role) notFound();
  return <RoleView role={role} />;
}
```

S4. Contact details are easy to scrape

What I saw: `opportunities@cynbit.in`, `support@cynbit.in` and three phone numbers are written as plain text on every page.

Why it matters: Bots collect these and the addresses get spam and phishing. The `opportunities@` address is the one candidates are told to use.

Fix: Keep the details on the contact page only, and encourage people to use the form. If an email must stay visible, load it with a small script or show it as an image-free obfuscated link, and make sure the mail server has spam filtering.

S5. Submission portal is not rated by a browser security tool

What I saw: When I opened the submission portal, a browser security extension showed a notice saying it has not assessed the site's safety yet and to enter personal info only if I trust it.

Why it matters: This is not a malware warning. It only means the security tool has no rating for the site. Still, candidates are asked to submit their work on a page their security tools have not rated, and that makes it look like a phishing page. It also teaches people to click through warnings, which is a poor habit for a company that sells security services.

Possible reasons (not confirmed): the portal is on a third-party domain instead of a cynbit.in address, the domain is new, the link goes through redirect domains, or there is a certificate issue.

Fix:

- Host the portal on a Cynbit subdomain such as `apply.cynbit.in` with a valid certificate and HSTS.
- Make sure the sender domain of the invitation email passes SPF, DKIM and DMARC.
- Add a `security.txt` file so people know where to report problems.
- Use short-lived signed links instead of long-lived tokens in the URL, and send `Referrer-Policy: no-referrer` and `Cache-Control: no-store` on that page.

```
# https://www.cynbit.in/.well-known/security.txt
Contact: mailto:security@cynbit.in
Expires: 2027-12-31T00:00:00.000Z
Preferred-Languages: en

# expiring signed link for candidates
import hmac, hashlib, time, base64, os

SECRET = os.environ["LINK_SIGNING_KEY"].encode()

def make_token(candidate_id: str, ttl_seconds: int = 72 * 3600) -> str:
```

```
    expires = int(time.time()) + ttl_seconds
    msg = f"{candidate_id}.{expires}".encode()
    sig = hmac.new(SECRET, msg, hashlib.sha256).digest()
    return base64.urlsafe_b64encode(msg + b"." + sig).decode()

def check_token(token: str):
    raw = base64.urlsafe_b64decode(token.encode())
    msg, _, sig = raw.rpartition(b".")
    expected = hmac.new(SECRET, msg, hashlib.sha256).digest()
    if not hmac.compare_digest(sig, expected):
        return None
    candidate_id, expires = msg.decode().split(".")
    return candidate_id if int(expires) > time.time() else None
```

D1. Wrong canonical and og:url on pages

What I saw: On the contact and careers pages the `og:url` is `https://cynbit.in` (the home address) and not the page's own address. The careers page also repeats the home page's keywords. Links on the site use `www.cynbit.in` but the meta tags use `cynbit.in`, so the same site can be reached on two hostnames.

Why it matters: Search engines and social platforms can treat the pages as duplicates, and shared links may point to the wrong page.

Fix: Set the base URL once, give each page its own canonical URL, and redirect the non-www address to www with a 301.

```
// app/layout.tsx
export const metadata = {
  metadataBase: new URL("https://www.cynbit.in"),
  openGraph: { siteName: "Cynbit Technologies", images: ["/og-1200x630.png"] },
};

// app/contact/page.tsx
export const metadata = {
  alternates: { canonical: "/contact" },
  openGraph: { url: "/contact" },
};

// next.config.js (add inside module.exports)
async redirects() {
  return [{
    source: "/:path*",
    has: [{ type: "host", value: "cynbit.in" }],
    destination: "https://www.cynbit.in/:path*",
    permanent: true,
  }];
},
```

D2. Share image is an SVG

What I saw: `og:image` and `twitter:image` both point to `logo.svg`.

Why it matters: LinkedIn, Facebook and X do not show SVG images in link previews, so shared Cynbit links will have no picture.

Fix: Use a 1200 x 630 PNG or JPG (shown in the snippet under D1).

D3. Live-looking numbers may be fixed text

What I saw: The home page shows "Active now", "Efficiency 95%", "System Stability 99.9%", "98% Retention", "25k+ architects" and "1000+ certified architects".

Why it matters: If these are typed into the page and not coming from real data, they look like live monitoring but are not. For a company that sells security audits and cloud services, claims like these should be backed by real numbers.

What I would check: Open the Network tab and see whether any API call returns these values.

Fix: Either feed them from a real source, or mark them clearly as examples. A small status endpoint (see E1) can back the uptime claim.

P1. Client logo list repeated four times

What I saw: The client names (Marketing Addict, Evento, Urban Wardrobe and others) appear four times in the page output. This is most likely a scrolling marquee.

Why it matters: It makes the HTML larger than needed and repeats the same content for screen readers.

Fix: Write the list once in the HTML. Copy it with CSS or JavaScript for the loop and add `aria-hidden="true"` to the copy.

P2. Caching for careers and blog

What I noticed: Careers and blog content changes rarely, but I could not see the caching headers.

What I would check: Look at `Cache-Control` and `Age` on `/careers` and `/blog`.

Fix: Generate these pages statically and refresh them with ISR (for example `revalidate = 3600`, as shown in S3) or on demand when a role or post is published.

E1. No health or status endpoint found

What I noticed: I did not find a way to check whether the backend and database are healthy.

Why it matters: Without one, uptime claims cannot be backed by monitoring, and it is harder to catch failures early.

Fix:

```
// app/api/health/route.ts
export const dynamic = "force-dynamic";

export async function GET() {
  const start = performance.now();
  const dbOk = await db.$queryRaw`SELECT 1`.then(() => true).catch(() => false);
  return Response.json(
    { status: dbOk ? "ok" : "degraded", db: dbOk, latency_ms: Math.round(performance.now() - start) },
    { status: dbOk ? 200 : 503, headers: { "Cache-Control": "no-store" } }
  );
}
```

Connect this to an uptime monitor and log errors in a structured format so problems can be traced.

A1. Login cookie scope across products

What I noticed: Cynbit has several products that presumably have logins.

Why it matters: If login cookies are shared across all `.cynbit.in` subdomains, a problem in one subdomain (for example a vulnerable blog plugin) could affect all the products.

Fix: Use cookies with the `__Host-` prefix, keep each product on its own origin, and use short-lived tokens. In the longer term, use one central login service (OIDC) for all products.

5. What I would fix first

1. **Quick wins (about one day):** security headers, correct canonical and og tags, PNG share image, non-www to www redirect, hide `X-Powered-By`.
2. **Forms:** validation, honeypot and rate limit on the contact and newsletter endpoints.

3. **Candidate portal:** move it to a Cynbit subdomain, add SPF/DKIM/DMARC and `security.txt`.
4. **Short term:** save inquiries in a database and send emails through a queue, add double opt-in, add the health endpoint and an uptime monitor, and replace fixed metrics with real ones.
5. **Longer term:** one login service for all products, structured logging and tracing.

6. Limits of this review

This review is based only on public pages and on what a normal browser showed. I did not log in to any Cynbit product, call private APIs or run any scanner, and I did not capture or analyse API request and response payloads, so this report does not include sample payloads. The items marked "need to confirm" are risks that I could see signs of, but I could not prove from outside, and they should be checked by someone with proper access.